

ME2605 / DDA6204 · Neural Net Training · Fall 2026

Lecture 3: Depth and Normalization

Why deep networks are hard, and what can fix it

Ruoyu Sun

Today's Plan

- **Why Depth Is Hard**
 - probability analysis; the depth-width ratio
- **Normalization**
 - alternative methods, how it's embedded into NN

Learning Goals

By the end of this lecture you should be able to:

- Explain why a deep net is hard to train
- Explain possible ways to control signal propagation
- Explain how a normalization layer is embedded into NNs
- Write LayerNorm and RMSNorm

Review: One-Layer Result

Claim 2.1: the entries of $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ are independent random variables with

$$\mathbb{E}[W_{i,j}] = 0, \quad \text{var}[W_{i,j}] = \frac{1}{d_{\text{in}}}, \quad \text{and then}$$

- (i) $\mathbb{E}|\langle W_{i,:}, x \rangle|^2 = \frac{1}{d_{\text{in}}} \|x\|^2$, $\mathbb{E}\left(\frac{1}{d_{\text{out}}} \|Wx\|^2\right) = \frac{1}{d_{\text{in}}} \|x\|^2$ for any fixed x ;
- (ii) if x is a random vector independent of W , both hold with $\|x\|^2$ replaced by $\mathbb{E}\|x\|^2$.

Why Depth Hurts

Variance analysis, the depth–width ratio

Multi-Layer Result

- Bottou–LeCun initialization keeps the expected signal in a multi-layer net too.

Corollary 3.1: suppose the entries of $W_l \in \mathbb{R}^{d_l \times d_{l-1}}$ satisfy

$$\mathbb{E}[W_{l;i,j}] = 0, \quad \text{var}[W_{l;i,j}] = \frac{1}{d_{l-1}}, \quad l = 1, 2, \dots, L,$$

and are mutually independent. Then for any fixed x , with $d_x = d_0$ and $d_y = d_L$,

$$\frac{1}{d_y} \mathbb{E} \|W_L \cdots W_1 x\|^2 = \frac{1}{d_x} \|x\|^2.$$

Proof: Induction Step

Apply the one-layer claim at every layer. Each z_{l-1} is independent of W_l , thus

$$\frac{1}{d_l} \mathbb{E} \|z_l\|^2 = \frac{1}{d_{l-1}} \mathbb{E} \|z_{l-1}\|^2, \quad l = 1, 2, \dots, L,$$

$$\implies \frac{1}{d_y} \mathbb{E} \|W_L \cdots W_1 x\|^2 = \frac{1}{d_x} \|x\|^2.$$

RMS Versus Norm

Both measure one vector z in $\mathbb{R}^d$, and they differ by a factor of $\sqrt{d}$.

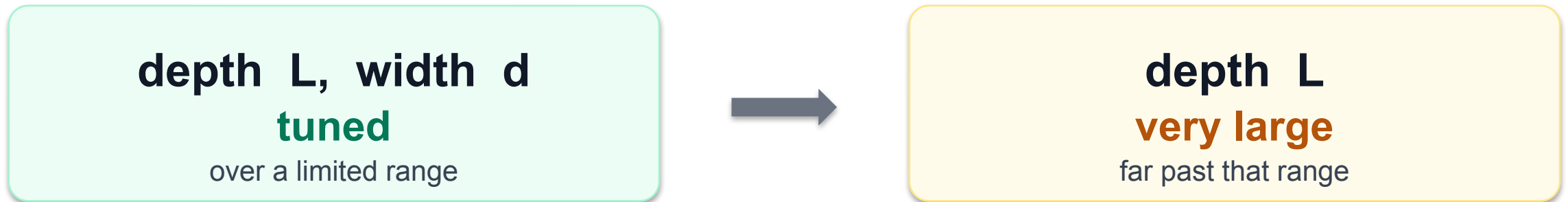
- **RMS (root mean square)** — the size of a typical entry.
- **norm (L2)** — the length of the whole vector.

$$\text{RMS}(z) = \sqrt{\frac{1}{d} \sum_{j=1}^d z_j^2}, \quad \|z\| = \sqrt{\sum_{j=1}^d z_j^2} = \sqrt{d} \text{RMS}(z).$$

- Signal propagation principle: hope $\text{RMS}(z_l) \approx 1$ at every layer, not for $\|z_l\| \approx 1$
(Sometimes we say “the norm of each layer output needs to be controlled”;
Technically, we mean “the RMS needs to be controlled”.)

Is He Initialization Enough for a Deep Net?

In the numerical experiment, we have tuned the width d .



What about the depth L ? We push L far past the range we tuned.

Does Depth Still Train?

We train a plain FCN at depths up to $L = 200$ and watch the training loss.

ruoyus.github.io/me2605/demos/deep-train-lab/

- **What you can change** — depth L , width d , initialization variance, activation.
- **What you see** — the per-layer scale; the loss curve
- **What is fixed** — two moons, mini-batch SGD, no bias.

Python: Depth 200, Width 10

Checking the signal propagation by python code:

deep linear net: L random matrices of size d by d , multiply input x .

Set $L = 200$ and $d = 10$.

```
import numpy as np
d = 10
L = 200
x0 = np.ones((d, 1))
x = x0
norm_x0 = np.linalg.norm(x0)
for l in range(L):
    W = np.random.randn(d, d)/np.sqrt(d)
    x = W.dot(x)
norm_x = np.linalg.norm(x)
ratio = norm_x/norm_x0
print(ratio)
```

- Over 12 runs the printed ratio ran from $6 \cdot 10^{-6}$ to $1.2 \cdot 10^{-2}$; the median was $6 \cdot 10^{-5}$.

Why is the output so small for a very deep net?

An Expectation Is Not a Typical Run

The expected ratio is around 1 for any depth; (by Corollary 3.1)

In a typical run, the ratio is very small; median ratio of $6 \cdot 10^{-5}$

Why?

**What is missing in the previous analysis:
variation.**

Multi-Layer: Two Assumptions

In a very deep net, the output signal is far from the input in terms of RMS.

To understand how the randomness of W makes $\|z_i\|$ vary,
we still consider [Bottou-LeCun initialization](#), for [deep linear net](#).

Assumption 3.1 (Gaussian weights): $W_{i,j} \sim \mathcal{N}(0, 1/d_{\text{in}})$, $i = 1, \dots, d_{\text{out}}; j = 1, \dots, d_{\text{in}}$.

Assumption 3.2 (equal input): $x_j = 1/\sqrt{d_{\text{in}}}$, $j = 1, \dots, d_{\text{in}}$.

Three Networks

Wide



$$d_{\text{in}} = 1, d_{\text{out}} = 1000$$

$$y = Wx$$

$$W_{ij} \sim \mathcal{N}(0, 1)$$

One neuron

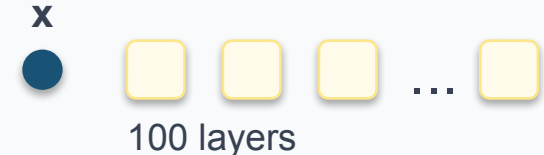


$$d_{\text{in}} = d_{\text{out}} = 1$$

$$y = wx$$

$$w \sim \mathcal{N}(0, 1)$$

Deep



100 layers

$$L = 100, d = 1$$

$$y = w_L \cdots w_1 x$$

$$w_l \sim \mathcal{N}(0, 1)$$

$M = (1/d_{\text{out}})\|y\|^2$ — the same quantity, with mean 1 in all three

Voting

For each network, guess how likely the output energy is to stay near its mean.

1) Wide network, $d = 1000$ — how large is $p = \Pr(0.8 < (1/d_{\text{out}})\|y\|^2 < 1.5)$?

2) One neuron, $d = 1$ — repeat the same question.

3) Deep network, 100 layers — repeat the same question.

A. $> 90\%$

B. $40\% \sim 90\%$

C. $5\% \sim 40\%$

D. $< 5\%$

Chebyshev Inequality

A few methods to estimate the tail probability.

One simple tool: Chebyshev inequality.

Lemma 3.1 (Chebyshev inequality). Let X be a real-valued random variable with $\mathbb{E}[X^2] < \infty$. Then for any $t > 0$,

$$\Pr\left(|X - \mathbb{E}[X]| \geq t\sqrt{\text{Var}(X)}\right) \leq \frac{1}{t^2}.$$

X is within t standard deviations of its mean, except with probability $\leq 1/t^2$.

Answers: the Wide Network

- 1) answer A. Here $M = (1/d)\|y\|^2 = (1/d)\sum_i y_i^2 = (1/d)\sum_i w_i^2$.
- This is a chi-square distribution scaled by $1/d$.
- For standard Gaussian distribution $X \sim N(0,1)$,

$$E[X^4] = 3, \text{ var}[X^2] = 2.$$

- Thus $\text{Var}(M) = (1/d^2)\sum_i \text{Var}(w_i^2) = (1/d) \text{Var}(w_1^2) = 2/d$.
- At $d = 1000$ the standard deviation is ~ 0.045 ;

Probability of M in $(0.8, 1.5) > 95\%$, by Chebychev inequality.

High-Dimensional Vectors Concentrate Near Sphere

This finding has a geometric implication.

- Given y with each $y_i \sim N(0,1)$, how likely is $\|y\|^2/d$ to be near 1?
- Answer: almost all Gaussian points sit near the sphere $\|y\|^2 = d$.
- Informal claim: $\|y\|^2 \in [d - C\sqrt{d}, d + C\sqrt{d}]$ with high probability.

Volume of a high-dimensional unit ball is concentrated near its surface.

Answers: the One Neuron and the Deep Network

- 2) answer C. Now $y = wx$, so $M = w^2$ — one draw of a chi-square.
- It lands in $(0.8, 1.5)$ with probability 15%; 63 % of draws fall below 0.8.
- 3) answer D. Here $y = w_{100} \dots w_1 x$, so $\log|y| = \sum_i \log|w_i|$.
- The mean of $\log|w|$ is ≈ -0.63 , so at $L = 100$ the sum is ≈ -63.5 .

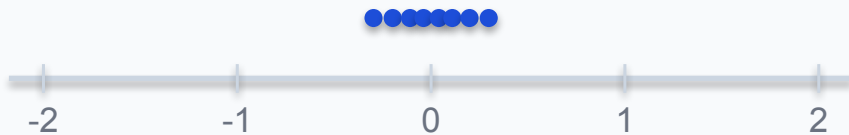
A typical output is $|y| \approx 10^{-27}$, and a typical M is $\approx 10^{-55}$.

Concentrate around Mean of Itself, or around Mean of Square?

In the one-neuron network, $y = wx$ is zero-mean with unit variance:

$$\mathbb{E}[y] = 0, \quad \mathbb{E}[y^2] = \mathbb{E}[y]^2 + \text{Var}(y) = 1.$$

Guess 1: concentrate around $\mathbb{E}[y] = 0$



Guess 2: concentrate around $\mathbb{E}[y^2] = 1$



**at most one of the two can be true:
concentrate around 0, or concentrate around 1.**

In fact, a standard Gaussian r.v. has more energy around 0, than around 1 and -1

Probability Analysis: 2-Layer

Next, we show some formal results.

Assumption 3.1 (Gaussian weights): $W_{i,j} \sim \mathcal{N}(0, 1/d_{\text{in}})$.

Assumption 3.2 (equal input): $x_j = 1/\sqrt{d_{\text{in}}}$.

Proposition 3.1

Suppose $y = Wx$, where $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and x satisfy Assumptions 3.1 and 3.2. Then

$$(i) \text{ Var}\left(\frac{1}{d_{\text{out}}} \|y\|^2\right) = \frac{2}{d_{\text{out}}}; \quad (ii) \text{ with probability at least 98\%, } \left| \frac{1}{d_{\text{out}}} \|y\|^2 - 1 \right| \leq \frac{10}{\sqrt{d_{\text{out}}}}.$$

Proposition 3.2

Suppose $y = Wx$, where $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and x satisfy Assumptions 3.1 and 3.2. Then

$$(i) \text{ Var}(M) = \frac{2}{d_{\text{out}}} \mathbb{E} \|x\|^2; \quad (ii) \mathbb{E}[M^2] = \left(1 + \frac{2}{d_{\text{out}}}\right) M_0^2; \quad (iii) \left| \frac{M}{M_0} - 1 \right| \leq \frac{10}{\sqrt{d_{\text{out}}}} \text{ w.p. } \geq 98\%.$$

$$\text{where } M = \frac{1}{d_{\text{out}}} \|y\|^2 \text{ and } M_0 = \frac{1}{d_{\text{in}}} \|x\|^2.$$

- Part (i) on the variance provides the high-probability bound (the deviation).
- Part (ii) on the 2nd moment is the key for the recursion.

Multi-Layer: Intuition

Let $x = z_{l-1}$, $y = z_l$ with $d_i = d$. Part (ii) of Proposition 3.2 gives

$$\mathbb{E}[\|z_l\|^4] = \left(1 + \frac{2}{d}\right) \mathbb{E}[\|z_{l-1}\|^4].$$

The fourth moment grows by a fixed factor at every layer.

The Depth–Width Ratio L/d

So, dropping the expectation notation,

$$\|z_L\|^4 \approx \|z_{L-1}\|^4 \left(1 + \frac{2}{d}\right)^L \approx \dots \approx \|x\|^4 \left(1 + \frac{2}{d}\right)^L \approx \|x\|^4 \exp\left(\pm \frac{2L}{d}\right).$$

This indicates that the depth–width ratio L/d is important for signal propagation.

Probability Analysis: Multi-Layer

Theorem 3.1 (signal propagation in a Gaussian deep linear net)

Suppose $y = W_L \cdots W_1 x$, where $W_l \in \mathbb{R}^{d_l \times d_{l-1}}$, $\forall l$, satisfies Assumption 3.1 and are independent.

Define $M_L = \frac{1}{d_{\text{out}}} \|y\|^2$ and $M_0 = \frac{1}{d_0} \|x\|^2$. Then

(i) $\mathbb{E}[M_L^2] = \prod_{l=1}^L \left(1 + \frac{2}{d_l}\right) M_0^2$;

(ii) with probability at least 99%, $\left| \frac{M_L}{M_0} - 1 \right| \leq 10\sqrt{\gamma}$, where $\gamma = \prod_{l=1}^L \left(1 + \frac{2}{d_l}\right) - 1$.

Corollary 3.2 (equal width, width–depth ratio $m/L = r > 1$)

Under the conditions of Theorem 3.1, suppose $d_0 = d_1 = \cdots = d_L = m$ and $m/L = r > 1$. Then w.p. at least 96%,

$$\left| \frac{M_L}{M_0} - 1 \right| \leq 5\sqrt{(1 + 2/m)^L - 1} = O\left(\frac{1}{\sqrt{r}}\right).$$

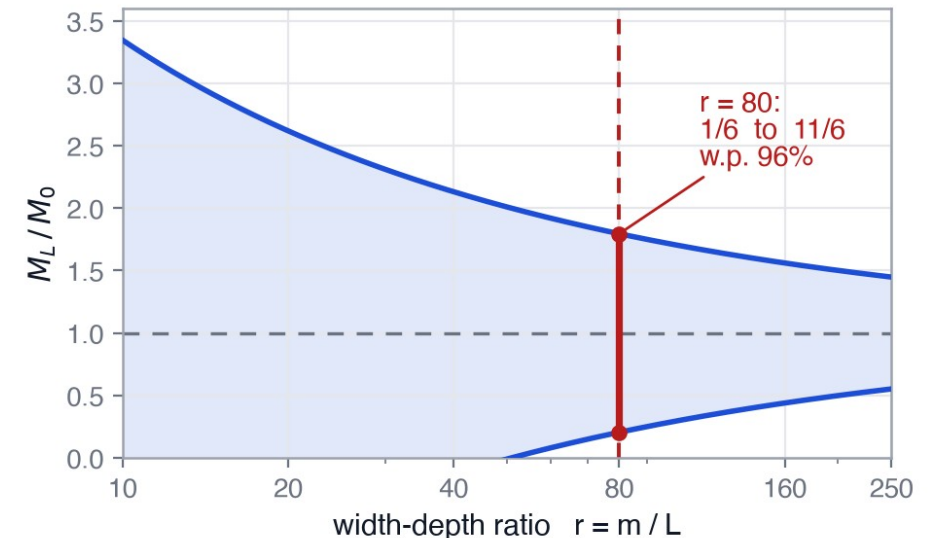
The Wide-Network Regime

Remark 3.1: when the width–depth ratio $m/L = r \geq 80$,

$$(1 + 2/m)^L - 1 = \left((1 + 2/m)^{m/2}\right)^{2/r} - 1 \leq e^{2/r} - 1 \leq e^{1/40} - 1 \leq 1/36,$$

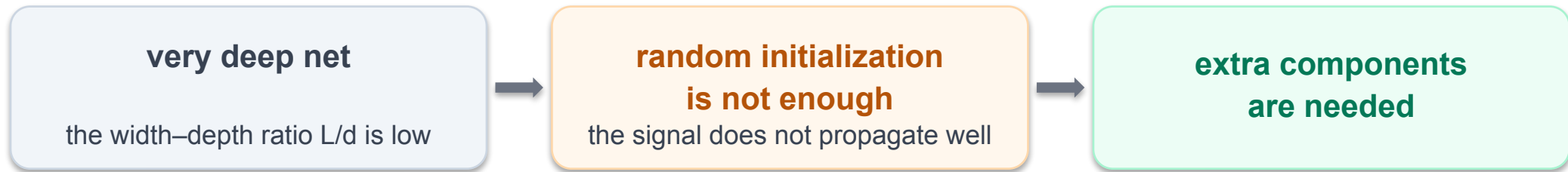
so with probability at least 96%, $\left|\frac{M_L}{M_0} - 1\right| \leq 5\sqrt{1/36} \leq \frac{5}{6}$, i.e. $\frac{1}{6} \leq \left|\frac{M_L}{M_0}\right| \leq \frac{11}{6}$.

- The bound above is loose: in practice the width–depth ratio does not have to reach 80 for the normalized output strength to sit in the range 1/6 to 2 with high probability.
- What matters is the order $1/\sqrt{r}$, and the intuition that a large width–depth ratio gives good signal propagation.



Extra Tricks Are Needed

What do we learn from the experiment and theory?



- **At this depth, the scaled random initialization is not enough.**
- Two families of fixes are used in practice.
 - **Classical fix** — normalization layer, ResNet.
 - **Alternative fix** — orthogonal initialization, etc.

Variance Analysis and Two Extreme Modes

Case 1 — Deep (narrow) net

FCN is “tall”; the width–depth ratio is low.

- width-10, depth-1000
- output below 10^{-20}

Mode 1: output near 0

behaves like a width-1, depth-100 net

Case 2 — Wide net

the width–depth ratio is high.

- width-100, depth-10
- output near 1

Mode 2: output near 1

behaves like a width-100, depth-2 net

Deep is bad for training! Wide is good!
(at least from the perspective of variance analysis)

Where the Signal Goes

The demo behind the rule:

per-layer pre-activation, post-activation and error.

ruoyus.github.io/me2605/demos/nn-signal-lab/

- **You can tune** — a gain g with $W \sim N(0, g^2/\text{fan}_{\text{in}})$; LeCun $g = 1$, He $g = \sqrt{2}$.
- **Task** — find the g that keeps every layer at one scale.

Signal Propagation: Deep Net Analysis

Ten-minute in-class quiz.

- **Format** —multiple-choice questions
- **Covered** — energy principle, one-layer claim, initializations, depth–width.
- **Time** — about 10 minutes, open book; every number is checkable by hand.

BREAK

Break



10 minutes

Normalization Layer

Neural-net Native Solution for Signal Control

Two Limits of Initialization

Draw W once; hope signals are controlled at each layer and each iteration.

the control is indirect

we set this



we hope for this



and it is only at step 0



- **Indirect** — we set W , not z_l ;

the signal follows through it.

- **Only control initial point, not later iterations**

Idea: can we control z_l during training, directly?

Part 2.1 Various Methods for Control

Goal Set, Method Missing

We have set up a goal now.

- Compared to SP: goal is control signal strength.
- Main method: tuning the initialization scale.
- Here, the goal is to normalize post-activations.

What shall be the method?

A Toy Problem: Keep $\|w\|$ Near 1

Example: consider the unconstrained problem $\min_w F(w)$, where F is a complicated function such that vanilla GD does not work well.

- Assume: after diagnosis, you find $\|w\|$ may become very large or very small during GD.
- As a remedy, we hope to keep $\|w\|$ close to 1 during training.
- Let's stress: not just $\|w\| \leq 1$, but $\|w\|$ close to 1.

How would you enforce it? Name at least two ways.

How Would You Enforce It?

Q1: what are possible approaches? Give at least two.

A: projection

B: regularizer

C: constraint

D: reparameterize

Q2: for each way you name, what does it cost you?

Methods 1 and 2: Projection and Penalty

Method 1 is gradient projection onto the sphere.

(1) projection: $w_{t+\frac{1}{2}} = w_t - \alpha \nabla F(w_t), \quad w_{t+1} = w_{t+\frac{1}{2}} / \|w_{t+\frac{1}{2}}\|$

(2) regularizer: $\min_w F(w) + \lambda R(w), \quad R(w) = (\|w\|^2 - 1)^2$

Method 2: add a regularizer to encourage $\|w\|$ to be close to 1;
 λ balances the original objective and the regularizer.

Methods 3 and 4: Constraint and Reparameterization

Method 3 (new formulation): add a constraint to enforce this requirement, to obtain a new optimization problem.

This new problem can be solved by projection, penalty, augmented Lagrangian, etc..

$$(3) \text{ constraint: } \min_w F(w) \text{ s.t. } \|w\| = 1$$

$$(4) \text{ reparametrize: } \min_v F(v/\|v\|)$$

Method 4 (new formulation): reparametrize w as $v/\|v\|$.

What Each One Costs

(1) **Projection** — the set is non-convex; convergence is unclear.

(2) **Regularizer** — the term that was added is non-convex.

(3) **Constraint** — the feasible set is non-convex.

An extra operator is introduced, which may increase the difficulty of optimization.

All four are reasonable choices here;

All four share a worse issue

The Bigger Drawback: Representation Power

A common disadvantage of most approaches:

they are not equivalent to the original problem; different optimal value

- For Methods 1 and 3, the feasible region is much smaller than the original region $\mathbb{R}^d$.
- The same holds for Method 4.
- For Method 2, the added regularizer might distort the original optimal solution.

In a word, restricting to $\|w\| = 1$ hurts the representation power.

(Disclaimer: Technically, adding regularizer is not exactly about representation power)

Fix: Embed the Requirement and Keep Expressiveness

Simple Fix: adding an extra scalar α to Method 4, to recover the feasible region.

Method 5: embedding an operation while keeping expressiveness.

$$\min_{\alpha \in \mathbb{R}, v \in \mathbb{R}^d} F\left(\alpha \frac{v}{\|v\|}\right).$$

- embed the operation into the objective;
- keep the representation power.

It becomes more non-convex — not a big issue for neural nets (already non-convex).

Feasible Region and Optimal Value

Claim 3.1: Suppose F is a real-valued function on $\mathbb{R}^d$. Then

$\{w \in \mathbb{R}^d : \|w\| = 1\} = \{v/\|v\| : v \in \mathbb{R}^d, v \neq 0\}$, and after adding the scalar α ,

$\{\alpha v/\|v\| : \alpha \in \mathbb{R}, v \in \mathbb{R}^d, v \neq 0\} = \mathbb{R}^d$,

so $\min_{w \in \mathbb{R}^d} F(w)$ and $\min_{\alpha \in \mathbb{R}, v \neq 0} F(\alpha v/\|v\|)$ have the same global optimal value.

Proof sketch: for $w \neq 0$ take $v = w$ and $\alpha = \|w\|$;

for $w = 0$ take $\alpha = 0$.

Dropping the Scalar α

The scalar α is necessary.

The following example shows that two problems can be different.

Claim 3: without α the two values separate. Take $F(w) = \|w\|^2$:

$$\min_{w \in \mathbb{R}^d} \|w\|^2 = 0 \quad \text{but} \quad \min_{\|w\|=1} \|w\|^2 = 1.$$

Part 2.2

What a Normalization Layer Is

Extension to Neural Net

Goal: normalize each layer output z_l ;

Essence: view the normalization step as a layer and add it to the network.

- It plays a similar role as nonlinear activation, or as a pooling layer: so we call it “normalization layer” in this lecture
- It is differentiable in its input and in its parameters, so BackProp can be applied.

This is a **neural-net native solution:**

adding a layer (a differentiable operation)

& consistent with BackProp (since still unconstrained optimization)

Normalization Operation

- Normalization is a broad word.
- A common understanding: dividing by the norm, $\mathbf{a} \mapsto \mathbf{a}/\|\mathbf{a}\|$
- **Standardization**: subtract the mean, divide by the standard deviation

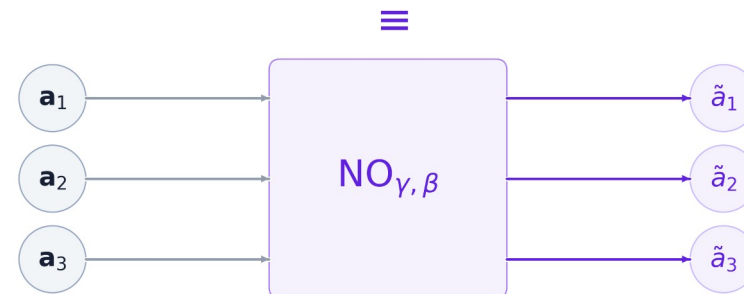
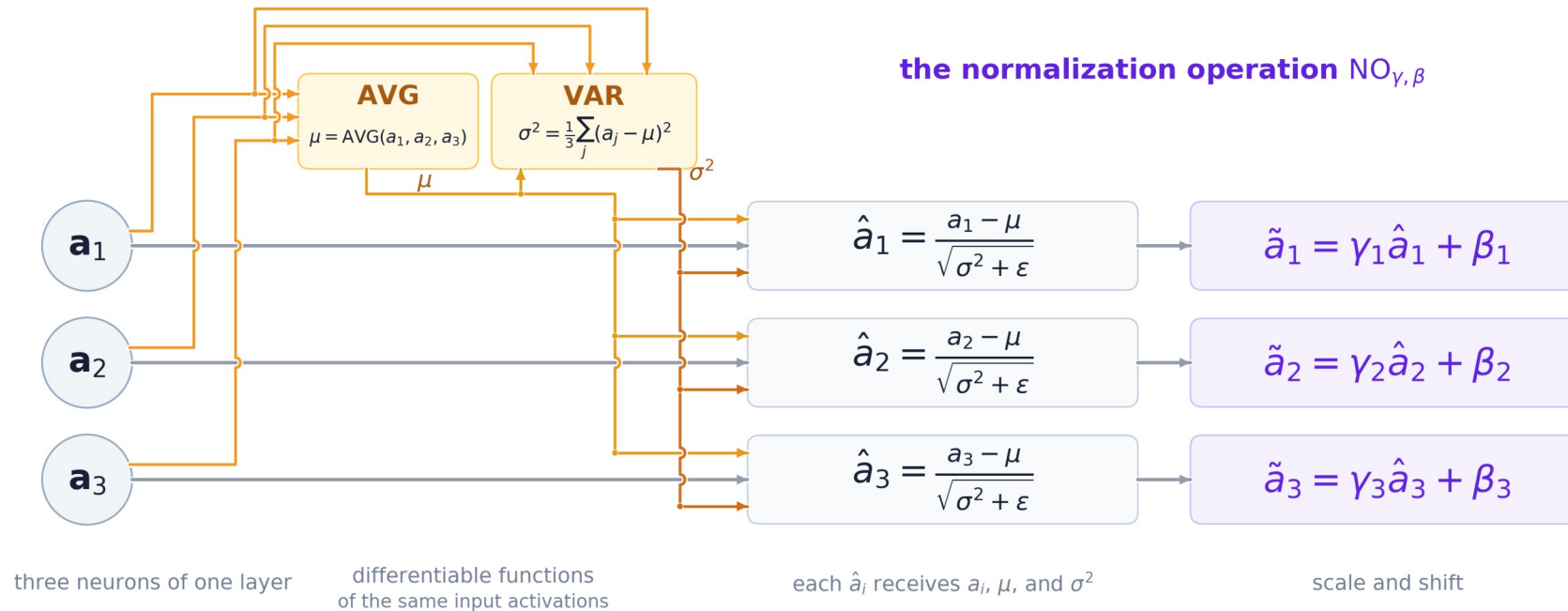
$$\text{NO}_{\gamma,\beta}(a_1, \dots, a_N) = \left(\gamma_1 \frac{a_1 - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta_1, \dots, \gamma_N \frac{a_N - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta_N \right), \quad \mu = \frac{1}{N} \sum_{i=1}^N a_i, \quad \sigma^2 = \frac{1}{N} \sum_{i=1}^N (a_i - \mu)^2.$$

γ and β are learned, one pair per feature; ε is a fixed small constant, say, 10^{-5} .

The operation is a mapping from
input $a_1, \dots, a_N$ to the output $\text{NO}(a_1, \dots, a_N)$.
It is differentiable w.r.t. the input $\mathbf{a}$ as well as the parameters γ, β .

Illustration of NO as a computation graph

We use a computation graph to illustrate NO.



Representation Power Kept for One Sample

Notation: $\text{NO}_{\gamma,\beta}(a) = \gamma \odot \hat{a} + \beta$, with one gain and one shift per feature: $\gamma, \beta \in \mathbb{R}^N$.

Claim 3.2: Suppose $a \in \mathbb{R}^N$, and write $\hat{a} = (a - \mu) / \sqrt{\sigma^2 + \varepsilon}$. Then:

- (i) $\sum_{i=1}^N \hat{a}_i = 0$ for every a .
- (ii) $\{\text{NO}_{\gamma,\beta}(a) : a \in \mathbb{R}^N, \gamma, \beta \in \mathbb{R}^N\} = \{a : a \in \mathbb{R}^N\} = \mathbb{R}^N$.

- In words: the output space is exactly the input space

More on representation power in the next lecture.

NO on a One-Sample Two-Layer Net

Start with the smallest setting: one sample, one hidden layer.

the plain net: $y = W_2 \phi(W_1 x) = f(x; W_2, W_1),$

with the operation: $y = W_2 \text{NO}_{\gamma, \beta}(\phi(W_1 x)) = f(x; W_2, \gamma, \beta, W_1).$

The set of learnable parameters becomes $(W_2, \gamma, \beta, W_1)$

Layer Normalization

Layer normalization: Normalizing each layer's output
We illustrate it using FCN, with one sample.

$$y = W_L \text{LN}_{\gamma_{L-1}, \beta_{L-1}} \left(W_{L-1} \cdots \text{LN}_{\gamma_1, \beta_1} \left(\phi(W_1 x) \right) \right),$$

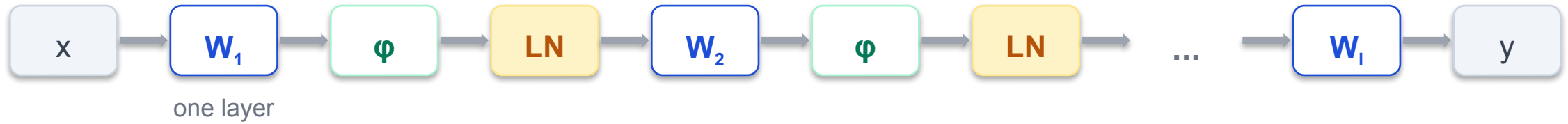
-- $\text{LN}_{\gamma, \beta} = \text{NO}_{\gamma, \beta}$: we call it LN, because NO is applied to **each layer's output**

--NO can be applied to other parts of the neural net: then it's not called LN, but other "xxx-normalization" (next time)

LN: a standard component of the Transformer.

LayerNorm in a Multi-Layer Network

An FCN with LN takes one input sample x and produces one prediction; set $\tilde{z}_0 = x$.



$$z_l = \phi(W_l \tilde{z}_{l-1}) \in \mathbb{R}^{d_l \times 1}, \quad l = 1, \dots, L-1,$$

$$\tilde{z}_l = \text{LN}_{\gamma_l, \beta_l}(z_l) \in \mathbb{R}^{d_l \times 1}, \quad l = 1, \dots, L-1,$$

$$y = W_L \tilde{z}_{L-1} \in \mathbb{R}^{d_L \times 1}.$$

After or Before Activation?

- Post-activation LN:
- Pre-activation LN: normalize before the activation;

in LLMs (transformers), there are specific types of LN, which we discuss later.

In short: post-activation LN is $z \leftarrow \text{LN}(\phi(Wz))$; pre-activation LN is $z \leftarrow \phi(\text{LN}(Wz))$.

$$\text{Post-activation LN: } z_l = \text{LN}_{\gamma_l, \beta_l}(\phi(W_l z_{l-1}))$$

$$\text{Pre-activation LN: } z_l = \phi(\text{LN}_{\gamma_l, \beta_l}(W_l z_{l-1}))$$

RMSNorm: Drop the Centering

RMSNorm: do not consider the mean.

(typical understanding of normalization)

RMSNorm is a mapping from the input $a_1, \dots, a_N$ to the output $\text{RMSNorm}(a_1, \dots, a_N)$; it is differentiable w.r.t. the input a as well as γ .

Why RMSNorm is better than LN in LLMs seems an open question.

$$\tilde{z}_l = \text{RMS}_{\gamma_l}(z_l) = \gamma_l \odot \frac{z_l}{\sqrt{\sigma_l^2 + \varepsilon}}, \quad \gamma_l \in \mathbb{R}^{d_l}, \quad \sigma_l^2 = \frac{1}{d_l} \|z_l\|^2, \quad l = 1, \dots, L-1,$$

No μ and no β : one reduction and one gain per feature.

Used commonly in LLMs

(when ppl say LLM use LayerNorm, they may mean RMSNorm)

SUMMARY

Summary

Depth and normalization — what to take away

Summary of Lec 3

Very deep net training issue: signal propagation is a challenge

- A specific run is different from the expectation; the variance grows with depth
- Variance depends on depth-width ratio L/d :
large depth makes the signal propagation bad

Normalization layer: controlling the forward signal

- Embedded layer of a network
- LayerNorm & RMSNorm: commonly used in language models
- Two forms: pre-activation LN and post-activation LN

END

Questions

Thank you.

Slides and HTML notes are posted with the course material.